A short review of relevant calculus concepts

Anand Narasimhamurthy

Numerical optimization in practice

- Numerical optimization problems arise in many practical applications such as Machine Learning, Operations Research, finance, engineering, experimental design and so on.
- Numerical optimization problems typically involve minimizing (or maximizing) a function of one or more variables.
  - The function which we seek to minimize (or maximize) is called the **objective function**
    Typically, a practical objective (eg. minimizing cost, maximizing throughput etc.) is expressed mathematically as a function of relevant variables.
  - In Operations Research or similar domains, the variables are called **decision variables**

Numerical optimization in practice

- Numerical optimization problems arise in many practical applications such as Machine Learning, Operations Research, finance, engineering, experimental design and so on.
- Numerical optimization problems typically involve minimizing (or maximizing) a function of one or more variables.
  - The function which we seek to minimize (or maximize) is called the **objective function**
    Typically, a practical objective (eg. minimizing cost, maximizing throughput etc.) is expressed mathematically as a function of relevant variables.
  - In Operations Research or similar domains, the variables are called **decision variables**

Numerical problems in machine learning

- In machine learning methods, typically,
  - the **loss (or cost) function** is the **objective**
  - the **model parameters** are the **variables of interest**
- In some cases (eg. Linear Regression, Logistic regression) the loss function is expressed explicitly as a function of the model parameters.
- In more sophisticated methods (eg. neural networks), it may not be feasible to express the cost function as an expression of all model parameters directly

Unconstrained and constrained optimization problems

- If the optimization problem only involves minimizing (or maximizing) an objective function $f(x_1, \ldots, x_d)$, then it is an **unconstrained optimization problem**
- If in addition to minimizing (or maximizing) the objective function $f(x_1, \ldots, x_d)$, some additional constraints need to be satisfied, then it is a **constrained optimization problem**

Review of background concepts

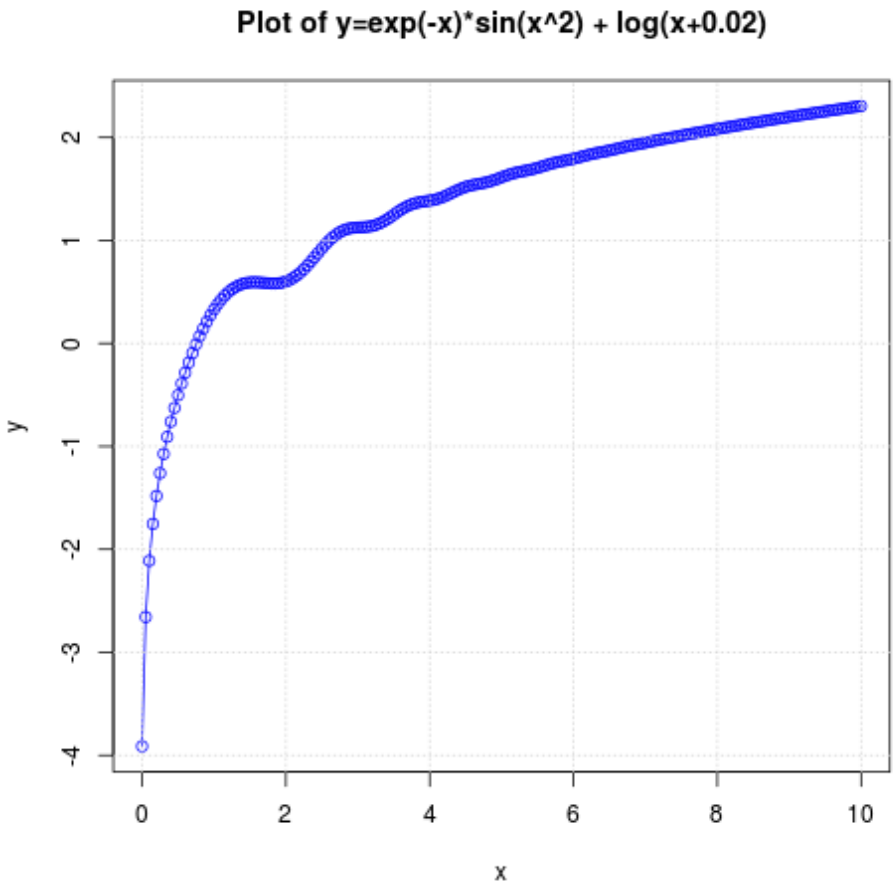
List of selected concepts

- Functions of a single variable and multi variable
- Derivative and partial derivatives
  - **Gradient**
  - **Chain rule** for differentiation
- Minimization of a function
- Gradient descent

Functions of a single variable and multi variable

Function of a single variable

A function of a single variable (say x) is denoted as $f(x)$. Examples :
$f(x) = e^x, f(x) = sin(x), f(x) = sin(x^2), f(x) = tanh(x)$ etc.
One advantage of a function of a single variable is that no matter how complicated the expression looks, the function can be visualized on a 2D plot. The figure below plots the function $f(x) = e^{-x} sin(x^2) + log(x + 0.02)$



Functions of multiple variables

In most real applications, it is common to have a function of multiple variables. Suppose these variables are named $x_1, x_2, ..., x_d$. The following are two common notations for a function of multiple variables.

- $f(x_1, x_2, \ldots, x_d)$
  This makes it more explicit that it is a function of d variables
- $f(x)$ where x is a vector i.e. $x = [x_1, x_2, \ldots, x_d]^T$
  A much more compact notation, useful especially when there are a large number of variables. Also most practical numerical computation approaches work with matrices and vectors.

Functions of multiple variables

In most real applications, it is common to have a function of multiple variables. Suppose these variables are named $x_1, x_2, ..., x_d$. The following are two common notations for a function of multiple variables.

- $f(x_1, x_2, \ldots, x_d)$
  This makes it more explicit that it is a function of d variables
- $f(x)$ where x is a vector i.e. $x = [x_1, x_2, \ldots, x_d]^T$
  A much more compact notation, useful especially when there are a large number of variables. Also most practical numerical computation approaches work with matrices and vectors.

Functions of multiple variables

Example : Consider the function $f(x_1, x_2) = x_1^2 + x_2^2$. Using matrix vector notation, this can be written as $f(\mathbf{x}) = x^T A x$, where $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ and $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

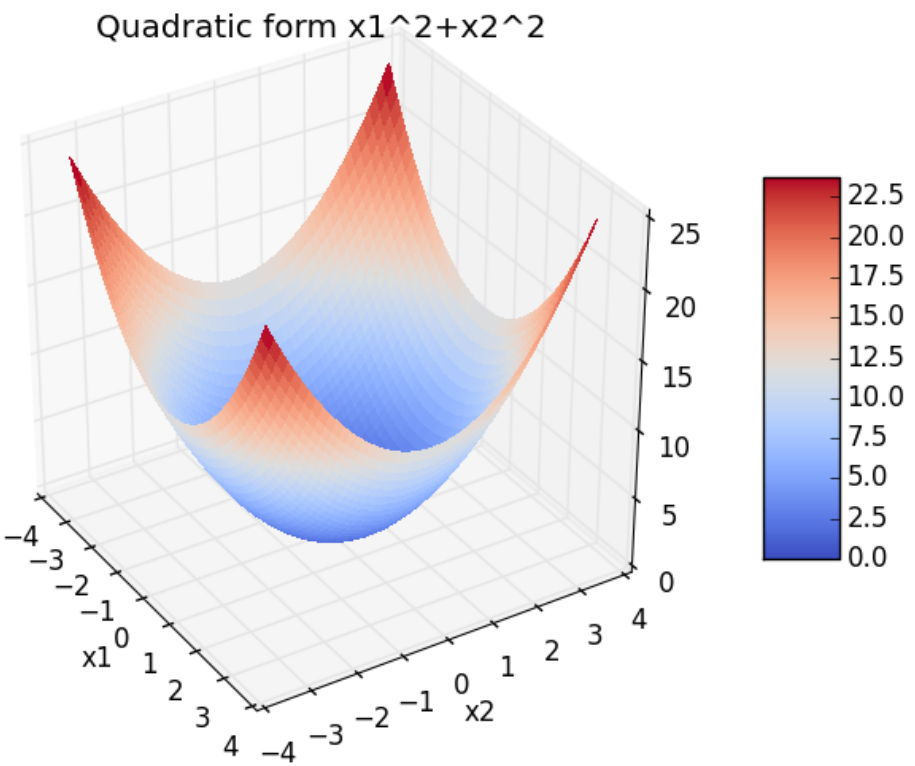


Figure 1: Plot of $f(x_1, x_2) = x_1^2 + x_2^2$

Derivatives and partial derivatives

Derivative of a function

- **Derivative :** The derivative of a function $f(x)$ with respect to the variable x can be interpreted as the rate of change of $f(x)$ with respect to x. Formally,

$$f'(x) = lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x))}{\Delta x}$$

Exercise Familiarize yourself with the derivatives of commonly used functions such as $sin(x), cos(x), e^x, x^p$ etc.

Partial derivatives

- **Partial derivative :** For a function of multiple variables i.e. say $f(x_1, \ldots, x_d)$, the partial derivative of the function with respect to a variable say x_i can be interpreted as the rate of change of $f()$ with respect to x_i, keeping all the other variables fixed.
- When taking the partial derivative wrt a variable (say x_1), we consider all other variables as constants.

Exercise Consider $f(x_1, x_2) = x_1 sin(x_2)$ Compute the partial derivatives wrt the independent variables.

Chain rule for differentiation

- **Function of one variable** Let $w = f(y)$ where $y = g(x)$,

$$\frac{dw}{dx} = \frac{dw}{dy}\frac{dy}{dx}$$

i.e.

$$\frac{dw}{dx} = f'(y)g'(x)$$

- **Function of multiple variables : Partial derivatives** Consider a function $w = F(x_1(t), x_2(t))$. Then

$$\frac{dw}{dt} = \frac{\partial w}{\partial_{x_1}}\frac{dx_1}{dt} + \frac{\partial w}{\partial_{x_2}}\frac{dx_2}{dt}$$

Sigmoid function

Consider the sigmoid function given by

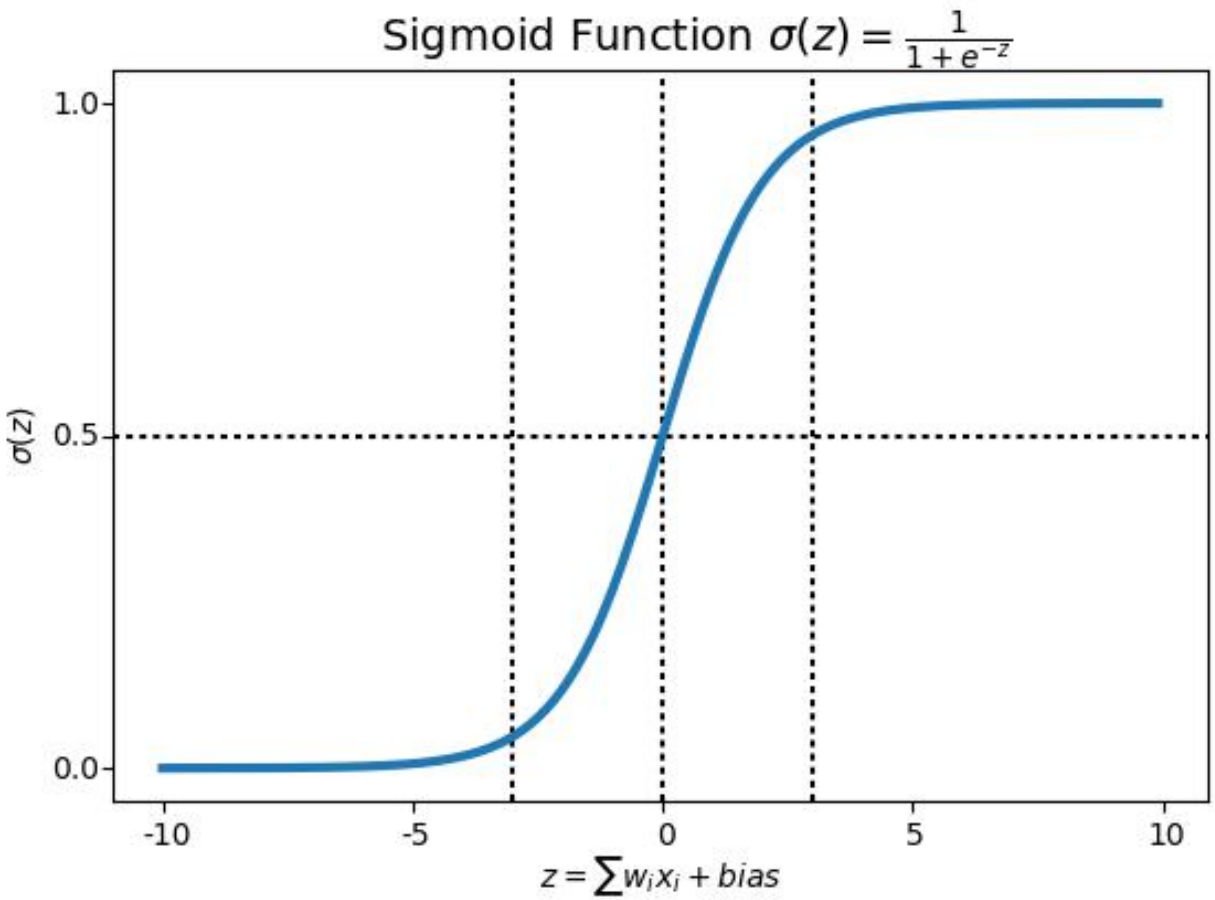
$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1}$$

You may remember seeing this in the discussion on logistic regression. Recall known facts about the sigmoid function.

- The value output by the sigmoid function is in the range $[0, 1]$ for any (real) value of the input. The sigmoid function can thus be considered as a **squashing function** i.e. it takes a value in the range $[-\infty, \infty]$ and outputs a value in the range $[0, 1]$

Sigmoid function

A plot of the sigmoid function is shown below.



Derivative of sigmoid function

Let us apply the chain rule to compute the derivative of the sigmoid function. $\sigma(z)$ can be written as $f(y)$ where

$$y = g(z) = 1 + e^{-z} \text{ and } f(y) = \frac{1}{y}$$

Note : This function composition is not unique, there could be multiple ways of expressing a function as a composition of functions. For example, $\sigma(z) = f(y) = \frac{1}{1+y}$ and $g(z) = e^{-z}$

Derivative of sigmoid function

Applying the chain rule to the function composition for the sigmoid function above, we get

$$
\begin{aligned}
\sigma'(z) &= f'(y)g'(z) \\
&= \frac{d}{dy}\left(\frac{1}{y}\right) \times \frac{d}{dz}(1+e^{-z}) \quad = \frac{-1}{y^2} \times -e^{-z} \\
&= \frac{e^{-z}}{(1+e^{-z})^2} \qquad \text{substituting back for } y \\
&= \frac{1}{1+e^{-z}} \frac{e^{-z}}{1+e^{-z}} \\
&= \sigma(z)(1-\sigma(z))
\end{aligned}
$$

since $\frac{e^{-z}}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = 1 - \sigma(z)$

The sigmoid and its derivative

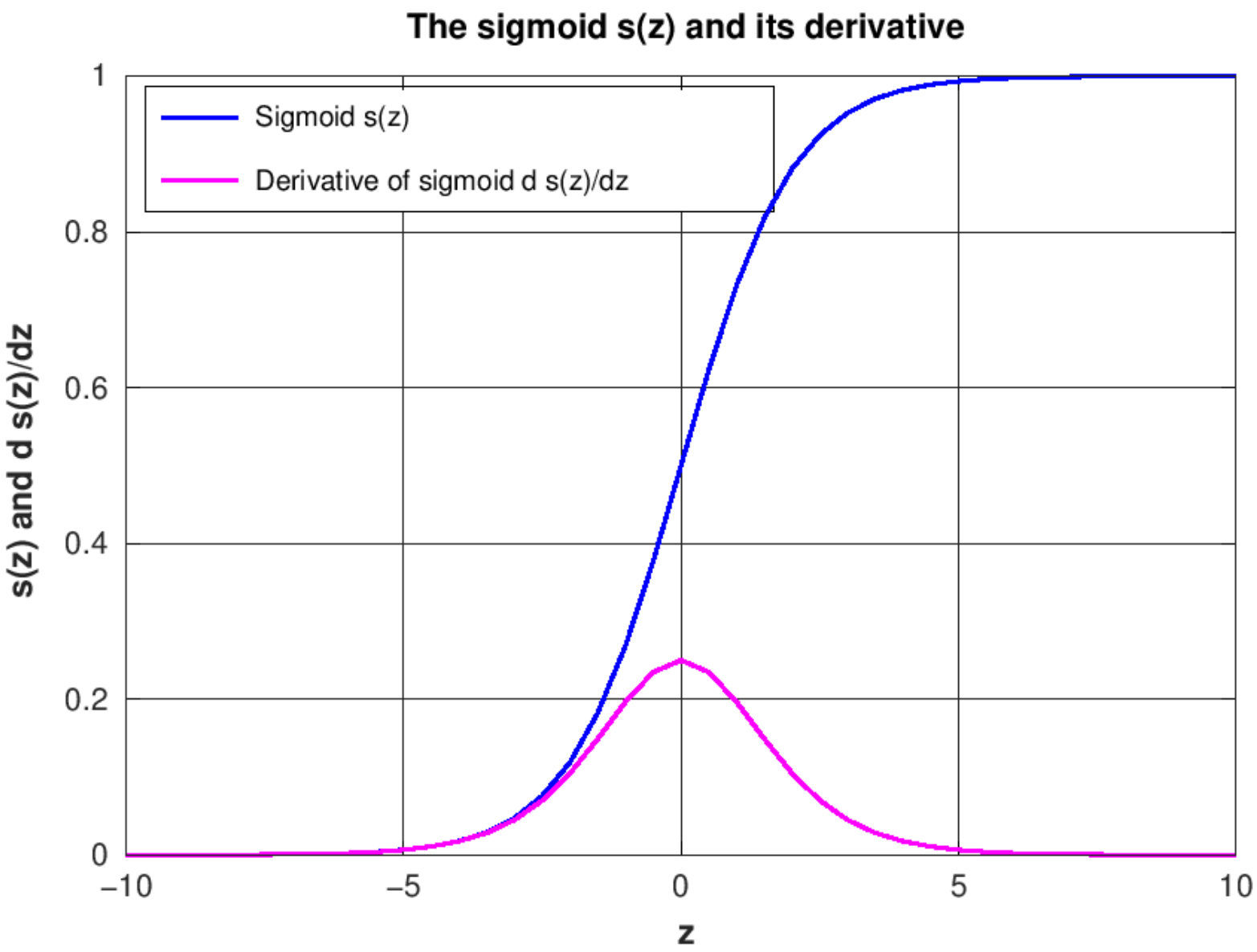


Figure 2: The sigmoid and its derivative

Exercise : Compute the **value** and the **derivative** of the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ at :

- $z = 5$
- $z = -1$
- $z = 0.5$
- $z = 0.05$

Gradient

Gradient of a function

Consider a function of multiple variables i.e. $f(x_1, x_2, \ldots, x_d)$. Another way of expressing the same is that f is a function of a vector $f(\mathbf{x})$ where $\mathbf{x} = [x_1, \ldots, x_d]^T$.

- The gradient can be considered as an extension of the concept of derivative for a function of multiple variables.
- When there are d variables, the rate of change (partial derivatives) can be computed with respect to each of the variables accordingly, the gradient is a vector of length d .

The gradient of the function with respect to $\mathbf{x}$ is defined as :

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_d} \end{bmatrix} \tag{2}$$

Gradient of a function

Consider a function of multiple variables i.e. $f(x_1, x_2, \ldots, x_d)$. Another way of expressing the same is that f is a function of a vector $f(\mathbf{x})$ where $\mathbf{x} = [x_1, \ldots, x_d]^T$.

- The gradient can be considered as an extension of the concept of derivative for a function of multiple variables.
- When there are d variables, the rate of change (partial derivatives) can be computed with respect to each of the variables accordingly, the gradient is a vector of length d .

The gradient of the function with respect to $\mathbf{x}$ is defined as :

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_d} \end{bmatrix} \tag{2}$$

Gradient of a function

Consider a function of multiple variables i.e. $f(x_1, x_2, \ldots, x_d)$. Another way of expressing the same is that f is a function of a vector $f(\mathbf{x})$ where $\mathbf{x} = [x_1, \ldots, x_d]^T$.

- The gradient can be considered as an extension of the concept of derivative for a function of multiple variables.
- When there are d variables, the rate of change (partial derivatives) can be computed with respect to each of the variables accordingly, the gradient is a vector of length d .

The gradient of the function with respect to $\mathbf{x}$ is defined as :

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_d} \end{bmatrix} \tag{2}$$

Example : Compute gradient of a function of multiple variables

Consider $f(x_1, x_2) = x_1^2 + 2x_2^2$. This will be used as a running example in many subsequent slides.
The gradient of this function is

$$\nabla_x f(x) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(x_1, x_2) \\ \\ \frac{\partial}{\partial x_2} f(x_1, x_2) \end{bmatrix}$$
$$= \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

Function minimization

Function minimization

Most machine learning/data fitting problems involve minimization of a function. Recall from calculus, the conditions for minimizing a function of one variable.

- **First order condition**
  First derivative is 0 i.e. $f'(x) = 0$
- **Second order condition**
  Let x^* be a stationary point i.e. a value where $f(x^*) = 0$. For x^* to be a local minimum, the second derivative i.e. $f''(x)$ (evaluated at x^*) must be positive.

Example : Minimize a function of one variable

Find the minimum value of the function $f(x) = x^2 - 10x + 4$

- Identify all stationary points. Taking the first derivative and equating to 0 gives, $2x - 10 = 0 \Rightarrow x = 5$
- Let x^* be a stationary point, i.e. a value of x where $f'(x) = 0$. For the function to achieve a minimum value at $x = x^*$, the second derivative should be positive at $x = x^*$
  The second derivative of above function is $f''(x) = 2$. Thus $f''(5) = 2$ i.e. the second derivative is positive at $x = 5$.
  Hence the given function achieves a local minimum at $x = 5$ (can be shown this is also the global minimum)

Note : The second derivative happens to not contain x in this specific instance but this may not be the case in general.

Minimization of a function of multiple variables

The equivalent first and second order conditions are :

- **First order condition** The gradient must be 0 (i.e. all components are 0) i.e. $\nabla_{\mathbf{x}} f(\mathbf{x}) = 0$
- **Second order condition** The Hessian matrix must be positive definite (dont worry about what the terms Hessian matrix and positive definiteness mean).

Example : Minimization of a function of multiple variables

Consider again the function $f(x_1, x_2) = x_1^2 + 2x_2^2$

- From an earlier slide we know that the gradient is, $\begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$ Equating it to 0 (i.e. equating all components separately to 0), we get $x_1 = 0, x_2 = 0$
- The Hessian computation and checking for positive definiteness are skipped here. It can be verified that the Hessian matrix for the above function is $\begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$ and is positive definite.

From the above, we can conclude that a **local minimum** occurs at $x_1 = 0, x_2 = 0$ ($f(0, 0) = 0$)

Note : For this function it can be shown that this is also the **global minimum**

Example : Minimization of a function of multiple variables

Consider again the function $f(x_1, x_2) = x_1^2 + 2x_2^2$

- From an earlier slide we know that the gradient is, $\begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$ Equating it to 0 (i.e. equating all components separately to 0), we get $x_1 = 0, x_2 = 0$
- The Hessian computation and checking for positive definiteness are skipped here. It can be verified that the Hessian matrix for the above function is $\begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$ and is positive definite.

From the above, we can conclude that a **local minimum** occurs at $x_1 = 0, x_2 = 0$ ($f(0, 0) = 0$)

Note : For this function it can be shown that this is also the **global minimum**

Example : Minimization of a function of multiple variables

Consider again the function $f(x_1, x_2) = x_1^2 + 2x_2^2$

- From an earlier slide we know that the gradient is, $\begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$ Equating it to 0 (i.e. equating all components separately to 0), we get $x_1 = 0, x_2 = 0$
- The Hessian computation and checking for positive definiteness are skipped here. It can be verified that the Hessian matrix for the above function is $\begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$ and is positive definite.

From the above, we can conclude that a **local minimum** occurs at $x_1 = 0, x_2 = 0$ ($f(0, 0) = 0$)

Note : For this function it can be shown that this is also the **global minimum**

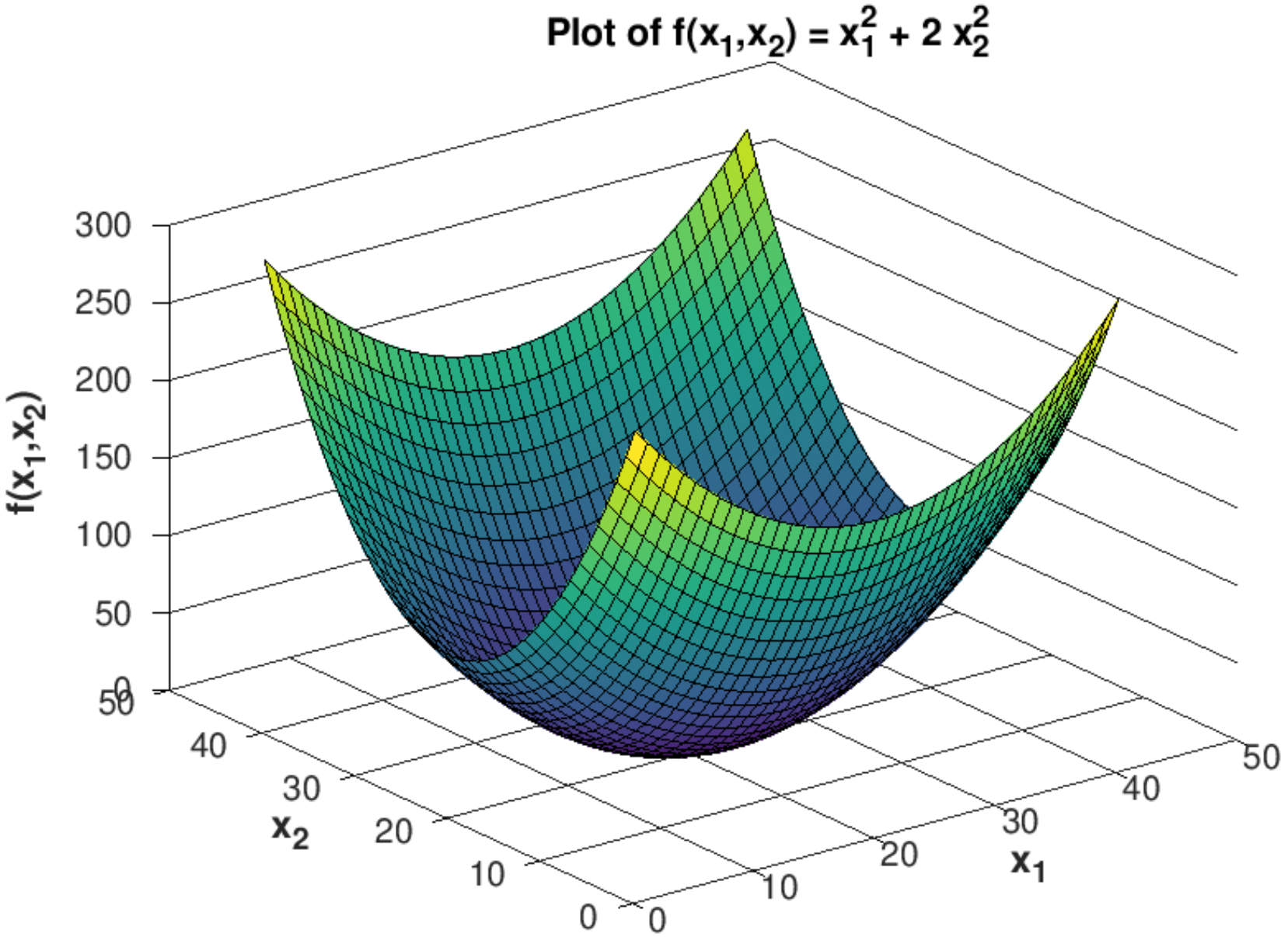


Figure 2: Surface plot of the function $f(x_1, x_2) = x_1^2 + 2x_2^2$

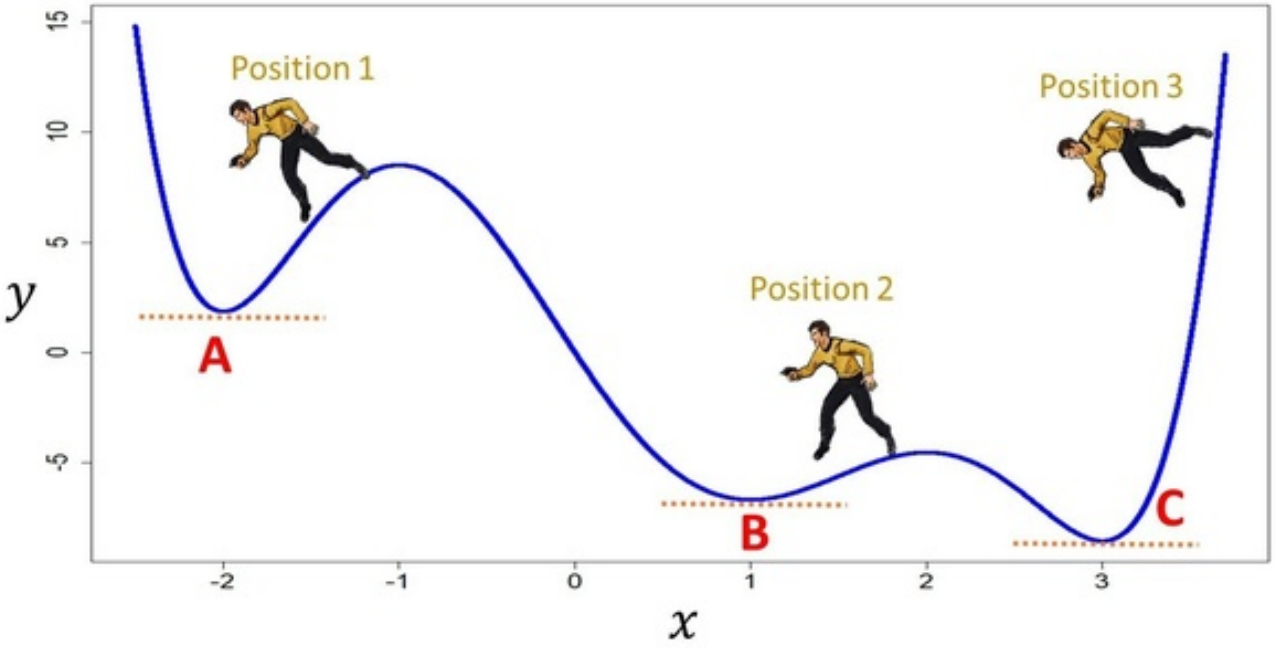
Minimization of an arbitrary function

- In some cases, (as in example on previous slide) it may be straightforward to apply the first and second order conditions directly to detect where (i.e. for what x) a local minimum is obtained for $f(x)$.
- In most cases, typically iterative methods are employed i.e. start with an initial guess $x^{(0)}$ and update till a local minimum is reached.
  Let $x^{(i)}$ denote the iterate at iteration i. To get the next iterate $x^{(i+1)}$, the **key questions are :**
  - Which direction to move next?
  - How large a step should one take?

Minimization of an arbitrary function

- In some cases, (as in example on previous slide) it may be straightforward to apply the first and second order conditions directly to detect where (i.e. for what x) a local minimum is obtained for $f(x)$.
- In most cases, typically iterative methods are employed i.e. start with an initial guess $x^{(0)}$ and update till a local minimum is reached.
  Let $x^{(i)}$ denote the iterate at iteration i. To get the next iterate $x^{(i+1)}$, the **key questions are :**
  - Which direction to move next?
  - How large a step should one take?

Minimization of an arbitrary function of one variable



Minimization of an arbitrary function

Two commonly used techniques

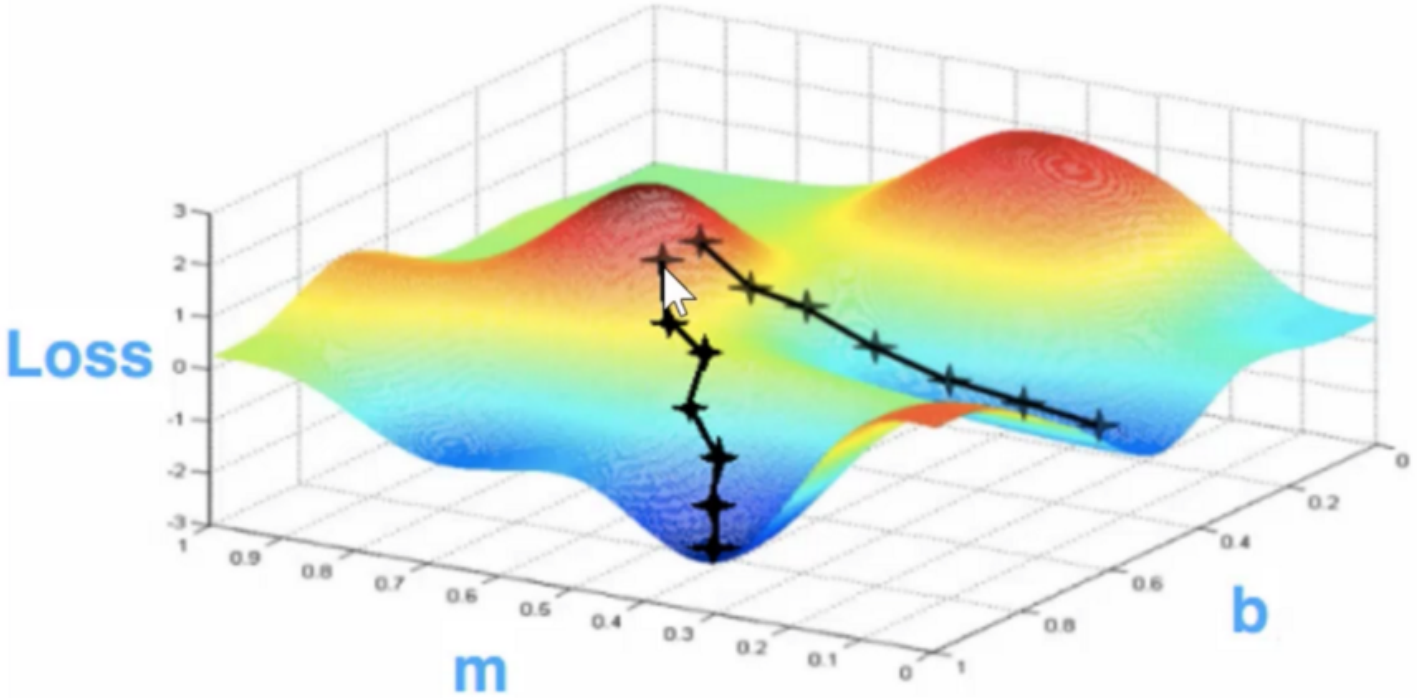
- **Gradient descent :** A first order method, where the direction is the **negative of the gradient**
- **Newton's method :** A second order method in which the direction is computed using the gradient and the inverse of the Hessian.

Newton's method (and variants) usually provide convergence in far fewer iterations than gradient descent. However, the computation cost involved in each iteration is more since the inverse of the Hessian needs to be computed.

Gradient descent

Gradient Descent

f (x) = nonlinear function of x



Gradient descent for function minimization : some theory

- From basic multivariate calculus, we know that a function can be approximated using only first order term of the Taylor's series expansion.
- Specifically, if we have a function $f(\vec{x})$ evaluated at $\vec{x} = \vec{x_0}$, then

$$f(\vec{x_0} + \vec{h}) \approx f(\vec{x_0}) + g(\vec{x_0})^T \vec{h}$$

  where $g(\vec{x_0})$ is the gradient of the function evaluated at $\vec{x} = \vec{x_0}$ and $||\vec{h}||_2$ is sufficiently small.
- It can be easily seen (proof omitted here) that choosing $\vec{h}$ within a small neighbourhood of $\vec{x_0}$ decreases the function
  i.e. if $\vec{h} = \alpha \times (-g(\vec{x_0}))$ with α sufficiently small, then $f(\vec{x_0} + \vec{h}) < f(\vec{x_0})$
- This is the basic foundation for gradient descent approaches.

Gradient descent for function minimization : some theory

- From basic multivariate calculus, we know that a function can be approximated using only first order term of the Taylor's series expansion.
- Specifically, if we have a function $f(\vec{x})$ evaluated at $\vec{x} = \vec{x_0}$, then

$$f(\vec{x_0} + \vec{h}) \approx f(\vec{x_0}) + g(\vec{x_0})^T \vec{h}$$

  where $g(\vec{x_0})$ is the gradient of the function evaluated at $\vec{x} = \vec{x_0}$ and $||\vec{h}||_2$ is sufficiently small.
- It can be easily seen (proof omitted here) that choosing $\vec{h}$ within a small neighbourhood of $\vec{x_0}$ decreases the function
  i.e. if $\vec{h} = \alpha \times (-g(\vec{x_0}))$ with α sufficiently small, then $f(\vec{x_0} + \vec{h}) < f(\vec{x_0})$
- This is the basic foundation for gradient descent approaches.

Gradient descent for function minimization

A general form of the update equation in gradient descent can be written as :

$$x^{(i+1)} = x^{(i)} - \alpha^{(i)} g(x^{(i)})$$

where,

- $x^{(i)}$ is the vector in the ith iteration
- $\alpha^{(i)}$ is the step size in the ith iteration
- $g(x^{(i)})$ is the gradient evaluated at $x = x^{(i)}$ i.e. $g(x^{(i)}) = \nabla_{\mathbf{x}} f(\mathbf{x})|_{x=x^{(i)}}$
- $x^{(i+1)}$ is the updated vector obtained in iteration $(i+1)$

Gradient descent for function minimization

Consider again the function $f(x_1, x_2) = x_1^2 + 2x_2^2$. We have already seen that this function can be minimized using the first and second order conditions directly.
Let us nevertheless apply gradient descent to minimize the function.

- Let a random initial guess be $x^{(0)} = \begin{bmatrix} 1 \\ 0.8 \end{bmatrix}$
- Let us use a fixed learning rate (say $\alpha = 0.05$)
- From earlier slides we know the gradient is $\begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$
- The update can be computed as :

$$x^{(1)} = \quad x^{(0)} - \alpha \nabla_{\mathbf{x}} f(\mathbf{x})|_{x=x^{(0)}}$$

Gradient descent for function minimization

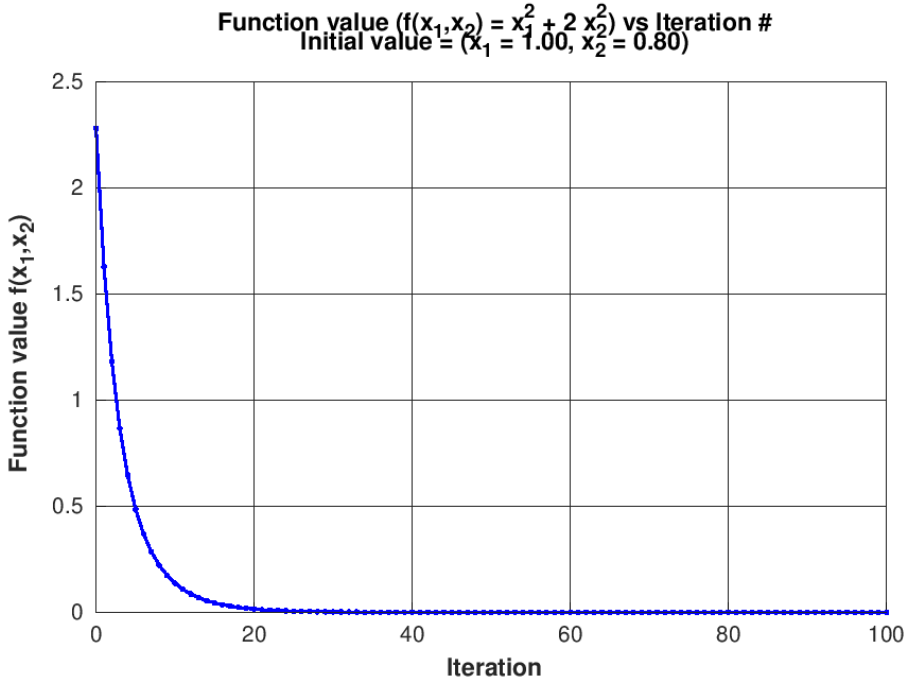
We illustrate the computations for a couple of iterations.

$$
\begin{aligned}
x^{(1)} &= \begin{bmatrix} 1 \\ 0.8 \end{bmatrix} - 0.05 \begin{bmatrix} 2 * 1 \\ 4 * 0.8 \end{bmatrix} \\
&= \begin{bmatrix} 1 - 0.05 * 2 \\ 0.8 - 0.05 * 3.2 \end{bmatrix} \\
&= \begin{bmatrix} 0.9 \\ 0.64 \end{bmatrix}
\end{aligned}
$$

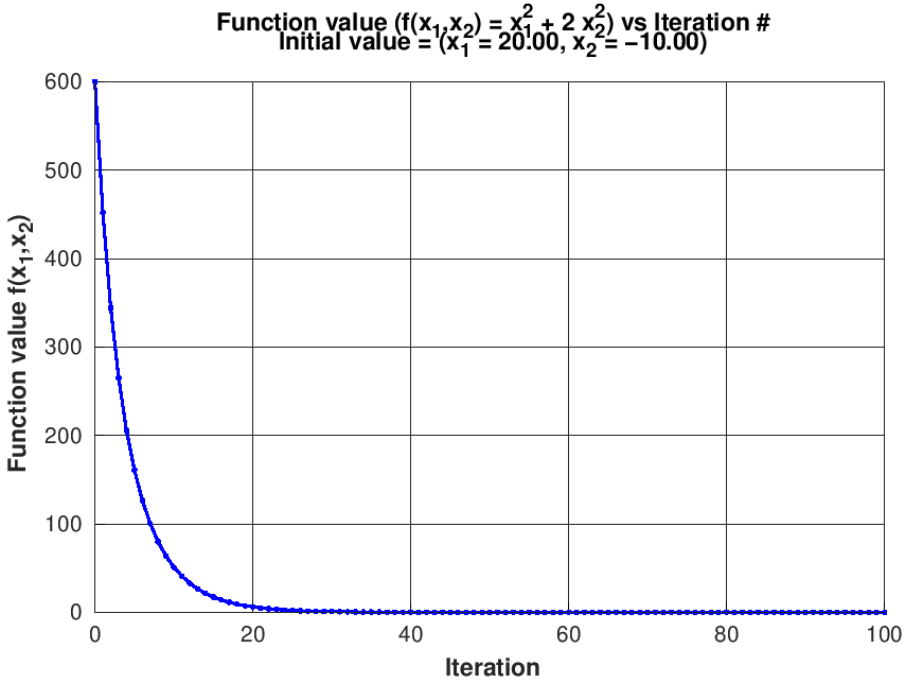
$$
x^{(2)} = \begin{bmatrix} 0.9 \\ 0.64 \end{bmatrix} - 0.05 \begin{bmatrix} 2 * 0.9 \\ 4 * 0.64 \end{bmatrix} = \begin{bmatrix} 0.81 \\ 0.512 \end{bmatrix}
$$

Function value vs iteration for different initializations

The gradient descent was run for 100 iterations, the function value is evaluated at the end of each iteration and plotted as shown in figure below.



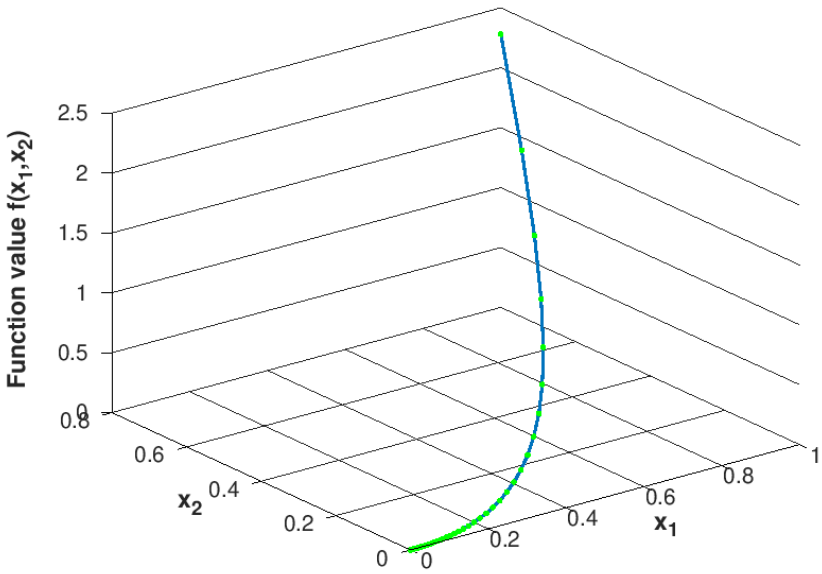
(a)



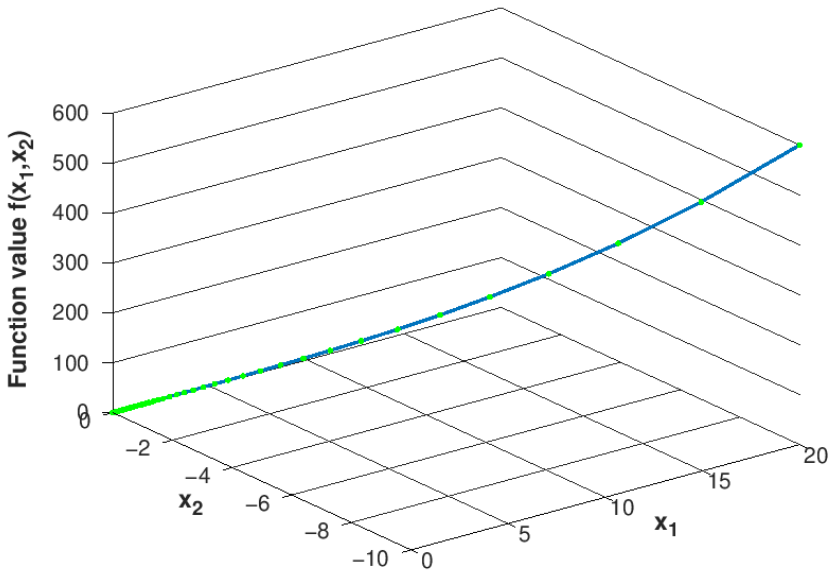
(b)

Figure 4: Initial guess (a) $x_1 = 1, x_2 = 0.8$ (b) $x_1 = 20, x_2 = -10$

Trajectory of the gradient descent



(a)



(b)

Figure 5: Initial guess (a) $x_1 = 1, x_2 = 0.8$ (b) $x_1 = 20, x_2 = -10$